TASK: 30

(OWASP Mobile Top 10)

Insufficient Cryptography and Poor Code Quality

&

Code Tampering and Reverse Engineering

Surendra Gabbani

Insufficient Cryptography and Poor Code Quality

What is Insufficient Cryptography?

Insufficient cryptography occurs when an application uses weak, outdated, or improperly implemented encryption methods to protect sensitive data. Instead of actually securing information, poor cryptography gives a *false sense of security* and makes it easy for attackers to decrypt or manipulate the data.

Examples

- Storing passwords using MD5.
- Encrypting data with a hardcoded AES key in the source code.
- Sending encrypted data without SSL/TLS (transport encryption missing).
- Base64 encoded data mistaken as encryption.

Impact:

- Data exposure
- Credential theft
- Reverse engineering the app
- Session hijacking
- Full compromise of user data

Prevention

- Use industry standards (AES-256, SHA-256, PBKDF2, bcrypt, Argon2)
- Secure key storage (Android Keystore, iOS Keychain)
- Apply proper salting/IV
- Avoid custom crypto
- Use HTTPS/TLS everywhere

What is Poor Code Quality?

Poor code quality refers to applications that are written in an unstructured, insecure, or inefficient manner. This type of code often contains bugs, unnecessary complexity, hardcoded values, weak input validation, or improper error handling.

Examples

- App crashing due to unhandled exceptions.
- Exposing sensitive data in logs.
- Hardcoded API keys or tokens.
- SQL queries built with string concatenation.
- Unoptimized and duplicated code.

Impact

- Increased attack surface.
- Vulnerabilities like XSS/SQLi/IDOR.
- App instability & crashes.
- Easier for attackers to reverse engineer.
- Higher maintenance cos.

Prevention

- Follow secure coding guidelines (OWASP, CERT).
- Conduct regular code reviews.
- Use static analysis tools (SonarQube, MobSF).
- Implement proper error handling.
- Remove unused/dead code.
- Validate all inputs.

Conclusion:

- Insufficient cryptography exposes sensitive data due to weak or incorrect encryption.
- Secure coding + strong cryptography = safer, stable applications.
- Regular testing, reviews, and standards are essential.
- Poor code quality increases security risks and reduces application reliability.

Code Tampering & Reverse Engineering

What is Code Tampering?

Code Tampering is the process of modifying an application's original code or resources after it has been developed and released.
Attackers tamper with the app to change how it behaves, disable security features, or inject malicious functionality.

Why attackers do it?

- Bypass login or authentication
- Disable root/jailbreak detection
- Unlock paid features for free
- Remove security controls
- Insert malware or malicious ads

How it happens:

- Attackers extract the APK/IPA → decompile it → modify the code → repackage and resign it → install or distribute the manipulated version.

Examples:

- Removing SSL pinning
- Changing app logic to skip login
- Enabling hidden debugging functions

Prevention:

- Strong obfuscation
- Code integrity checks
- Secure signature verification
- Use server-side validation
- Anti-tamper libraries

What is Reverse Engineering?

Reverse Engineering is the process of analyzing an application to understand its internal structure, logic, and code without having the original source.

Attackers do this to study how the app works, find weaknesses, or extract sensitive information

Why attackers do it:

- Read business logic
- Access hardcoded API keys, tokens, URLs
- Identify security flaws
- Bypass protections
- Understand data flow and encryption
- Prepare for code tampering

.

How it happens?

- They use tools to decompile, debug, or hook the application (e.g., JADX, Ghidra, Apktool, Frida) and inspect how it works behind the scenes

How Attackers Reverse Engineer Apps:

1. Extract APK
2. Decompile into readable SMALI/Java
3. Identify security checks (root, SSL pinning, auth logic)
4. Modify or patch the code
5. Repack & sign the APK
6. Install tampered app

Prevention Techniques:

- Code obfuscation
- Runtime integrity checks
- Root detection
- Anti-debugging

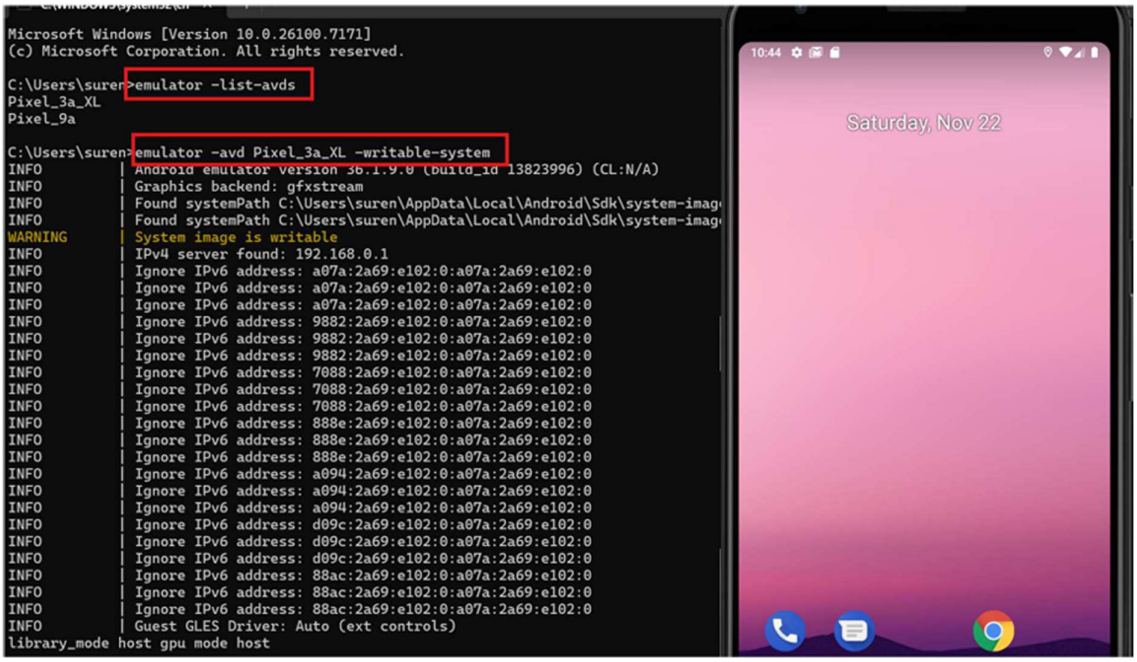
LAB PRACTICAL

Find the practical lab of Code tampering & Poor code Quality .

Lab APK: https://github.com/0xArab/diva-apk-file/blob/main/DivaApplication.apk

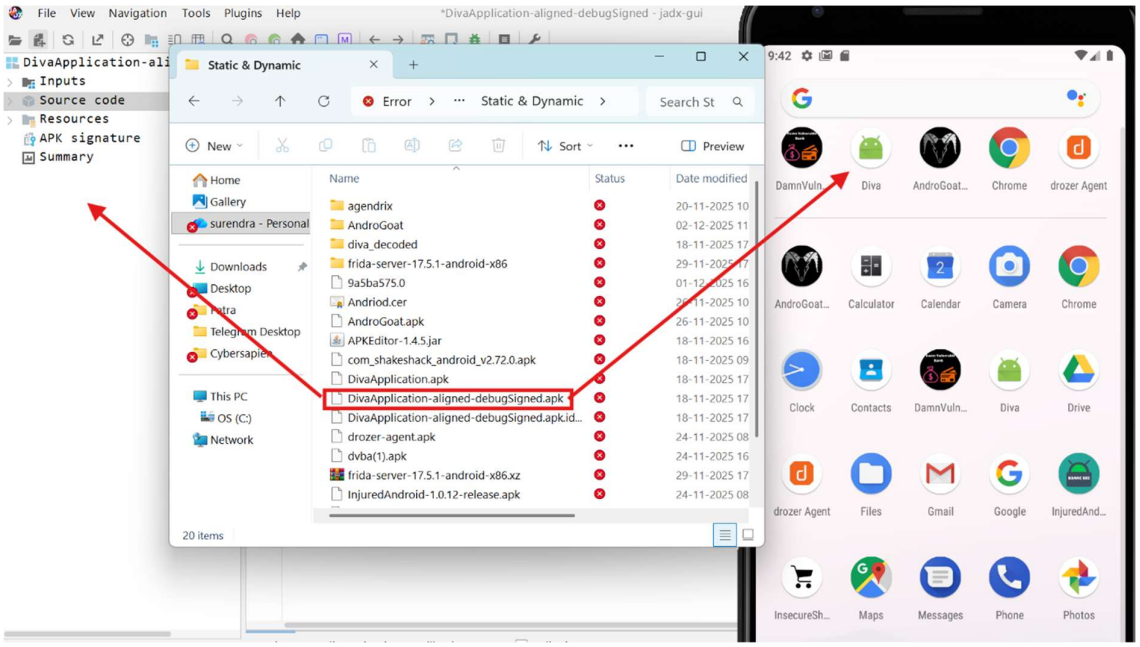
STEP 1:

- Open command prompt and list all the emulators (AVDs).
- Start emulator with writeable system by using commands as below.
- emulator -list-avds.
- emulator -avd Pixel_3a_XL -writable-system.



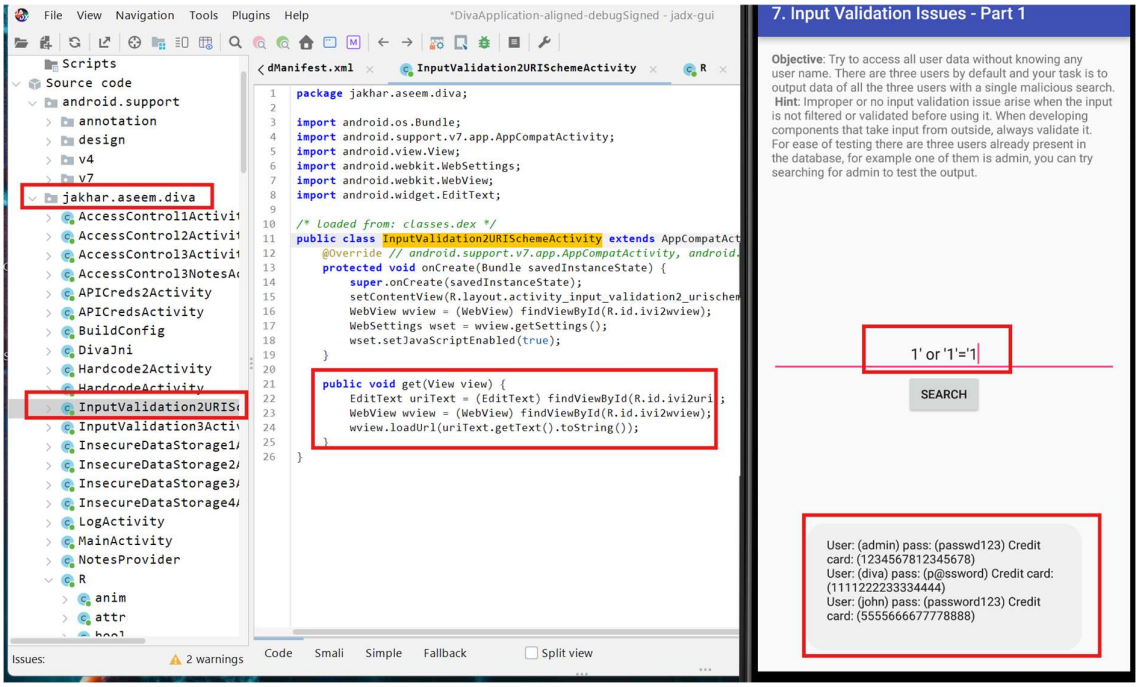
STEP 2:

- Download the APK Application.
- Decompile with the signature.
- Setup the Diva APK in both jadx and virtual emulator, by simply dragging into



STEP 3:

- View the source code of APK completely.
- Go through the folders of highlighted portion.
- Click andriodmanifest.xml view the source code of Input Validation 1.
- By giving the input of SQL queries we can expose the complete data.

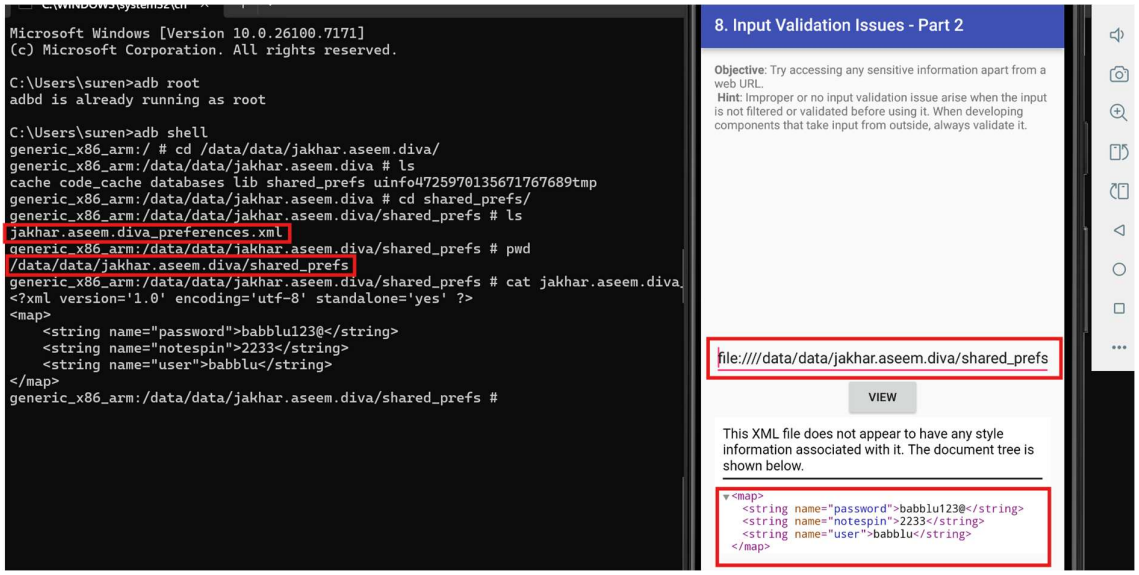


STEP 4:

Here need to view input validation part 2.

- Get root access in emulator and enter Android OS terminal shell.
- Move to app's private storage and list files in directory.
- View SharedPreferences folder in XML, read sensitive stored data user credentials below

file:////data/data/jakhar.aseem.diva/shared_prefs/jakhar.aseem.diva_preferences.xml



Conclusion:

Insufficient cryptography and poor code quality weaken the overall security of an application by exposing data, increasing vulnerabilities, and reducing system stability. By implementing strong encryption, following secure coding standards, and performing regular reviews and testing, organizations can significantly reduce risks and build more secure, reliable applications. Weak encryption and poor coding practices create major security gaps. Ensuring strong cryptography, clean coding, and continuous testing helps protect data and maintain a secure, stable application.

References:

- https://owasp.org/www-project-mobile-security-testing-guide/
- https://owasp.org/www-project-top-ten/
- https://csrc.nist.gov/publications
- https://wiki.sei.cmu.edu/confluence/display/seccode

THANK YOU